

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерных технологий

Лабораторная работа 5

**«Численное решение обыкновенных
дифференциальных уравнений»**

по дисциплине «Вычислительная математика»

Вариант 13

Выполнила:

Павличенко Софья Алексеевна, Р3215

Проверила:

Малышева Татьяна Алексеевна

Санкт-Петербург 2025г.

Оглавление

Цель	3
Программная реализация задачи	3
Листинг программы	3
Результат работы программы.....	6
Вывод	8

Цель

Решить задачу Коши для обыкновенных дифференциальных уравнений численными методами.

Программная реализация задачи

Листинг программы

```
import numpy as np
from matplotlib import pyplot as plt

MAX_ITERATIONS = 500

def euler_method(x0, y0, h, xn, p, solution):
    table = [(x0, y0)]
    x, y = x0, y0
    print(f"{x:<15.5f}{y:<20.8f}{solution(x):<20.8f}")
    step = euler_steps[p]
    iter_count = 0
    res_h = h

    while x + h - xn < 1e-9 and iter_count < MAX_ITERATIONS:
        y_h = step(x, y, h)
        y_h2 = step(x + h / 2, step(x, y, h / 2), h / 2)
        R = abs(y_h - y_h2) / (2 ** p - 1)

        if R < eps:
            x += h
            y = y_h
            table.append((x, y))
            res_h = h
            print(f"{x:<15.5f}{y:<20.8f}{y_h2:<20.8f}{solution(x):<20.8f}")
        else:
            print(f"\n! Погрешность R = {R:.7f} > eps => h = {h:.7f} -> {h / 2:.7f} !\n")
            h /= 2
            iter_count += 1
    if iter_count >= MAX_ITERATIONS:
        print("Превышено максимальное кол-во итераций!")
    return table, res_h

def milne_method(x0, y0, xn, h, solution):
    while True:
        euler_table, h = euler_method(x0, y0, h, x0 + 3*h, 2, solution)

        x = [p[0] for p in euler_table]
        y = [p[1] for p in euler_table]

        iter_count = 0
        failed = False

        while x[-1] + h - xn <= 1e-9 and iter_count < MAX_ITERATIONS:
```

```

        y_pred = y[-4] + (4 * h / 3) * (2 * f(x[-3], y[-3]) - f(x[-2], y[-2])
+ 2 * f(x[-1], y[-1]))
        x_new = x[-1] + h
        y_corr = y[-2] + h / 3 * (f(x[-2], y[-2]) + 4 * f(x[-1], y[-1]) +
f(x_new, y_pred))

        err = abs(y_corr - solution(x_new))

        if err < eps:
            x.append(x_new)
            y.append(y_corr)
            print(f"{x_new:<15.5f}{y_corr:<20.8f}{solution(x_new):<20.8f}")
        else:
            print(f"\n! Погрешность err = {err:.7f} > eps => уменьшаем h =
{h:.7f} -> {h/2:.7f} и пересчитываем !\n")
            h /= 2
            failed = True
            break

        iter_count += 1

    if iter_count >= MAX_ITERATIONS:
        print("Превышено максимальное кол-во итераций!")
    if not failed:
        return list(zip(x, y))

    if h < 1e-6:
        print("Шаг стал слишком мал!")
        return None

def plot(table, method_name):
    x_points = [x for x, _ in table]
    y_points = [y for _, y in table]

    x_exact = np.linspace(min(x_points), max(x_points), 500)
    y_exact = [solution(x) for x in x_exact]

    plt.figure()
    plt.plot(x_exact, y_exact, label="Точное решение", color='red', linewidth=2)
    plt.plot(x_points, y_points, 'o--', label=method_name, color='blue')
    plt.title(f"{method_name}")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

def print_header():
    header = f"{'x_i':<15}{ 'y_i':<20}{ 'Точное решение':<20}"
    print(header)
    print("-" * len(header))

# --- ФУНКЦИИ ОДУ ---

funcs = {
    '1': lambda x, y: y,

```

```

    '2': lambda x, y: -2 * x * y,
    '3': lambda x, y: x + y,
    '4': lambda x, y: np.cos(x) - y
}

solutions = {
    '1': lambda x: y0 / np.exp(x0) * np.exp(x),
    '2': lambda x: y0 / np.exp(-x0 ** 2) * np.exp(-x ** 2),
    '3': lambda x: (y0 + x0 + 1) / np.exp(x0) * np.exp(x) - x - 1,
    '4': lambda x: 0.5 * (np.sin(x) + np.cos(x) + ((y0 - 0.5 * (np.sin(x0) +
np.cos(x0))) / np.exp(-x0)) * np.exp(-x))
}

euler_steps = {
    1: lambda x, y, h: y + h * f(x, y),
    2: lambda x, y, h: y + h / 2 * (f(x, y) + f(x + h, y + h * f(x, y)))
}

# --- Ввод данных ---

print('Вычислительная математика. Лабораторная работа 6: "Численное решение
ОДУ".\n')

print('y\'= \n1: y\n2: -2xy\n3: x + y\n4: cos(x) - y')
func_number = input('Выберите ОДУ: ')

while func_number not in {'1', '2', '3', '4'}:
    func_number = input('Выберите первое (1), второе (2), третье (3) или
четвёртое (4) уравнение: ')

f = funcs[func_number]
solution = solutions[func_number]

print('Введите интервал дифференцирования:')
x0 = float(input('x_0 = '))
xn = float(input('x_n = '))
print()
y0 = float(input('y_0 = y(x_0) = '))
h = float(input('Шаг h = '))
eps = float(input('Точность ε = '))

# --- Решение задачи Коши ---

n = int((xn - x0) / h) + 1

print("\nМетод Эйлера")
print_header()
euler = euler_method(x0, y0, h, xn, 1, solution)[0]
print("\n\n")
plot(euler, "Метод Эйлера")

print("\nУсовершенствованный метод Эйлера")
print_header()
improved_euler = euler_method(x0, y0, h, xn, 2, solution)[0]
print("\n\n")
plot(improved_euler, "Усовершенствованный метод Эйлера")

print("\nМетод Милна")
print_header()

```

```
milne = milne_method(x0, y0, xn, h, solution)
plot(milne, "Метод Милна")
```

Результат работы программы

Вычислительная математика. Лабораторная работа 6: "Численное решение ОДУ".

```
y' =
1: y
2: -2xy
3: x + y
4: cos(x) - y
Выберите ОДУ: 2
Введите интервал дифференцирования:
x_0 = 0
x_n = 1
```

```
y_0 = y(x_0) = 1
Шаг h = 0.1
Точность ε = 0.01
```

Метод Эйлера

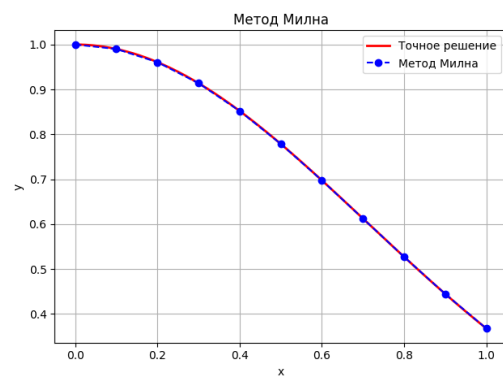
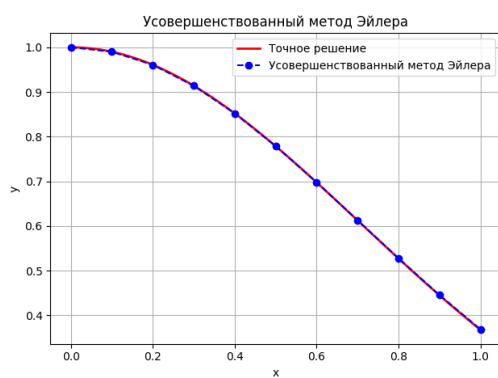
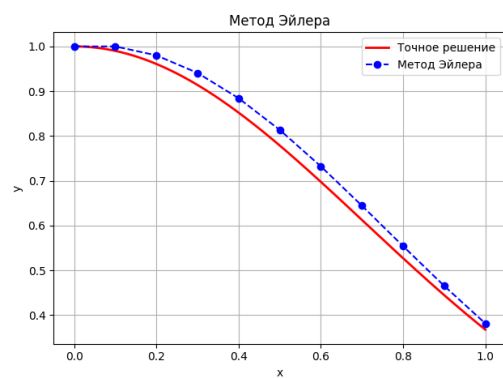
x_i	y_i	Точное решение
0.00000	1.000000000	1.000000000
0.10000	1.000000000	0.99004983
0.20000	0.980000000	0.96078944
0.30000	0.940800000	0.91393119
0.40000	0.88435200	0.85214379
0.50000	0.81360384	0.77880078
0.60000	0.73224346	0.69767633
0.70000	0.64437424	0.61262639
0.80000	0.55416185	0.52729242
0.90000	0.46549595	0.44485807
1.00000	0.38170668	0.36787944

Усовершенствованный метод Эйлера

x_i	y_i	Точное решение
0.00000	1.00000000	1.00000000
0.10000	0.99000000	0.99004983
0.20000	0.96069600	0.96078944
0.30000	0.91381404	0.91393119
0.40000	0.85204021	0.85214379
0.50000	0.77876475	0.77880078
0.60000	0.69777321	0.69767633
0.70000	0.61292399	0.61262639
0.80000	0.52785014	0.52729242
0.90000	0.44571666	0.44485807
1.00000	0.36905339	0.36787944

Метод Милна

x_i	y_i	Точное решение
0.00000	1.00000000	1.00000000
0.10000	0.99000000	0.99004983
0.20000	0.96069600	0.96078944
0.30000	0.91381404	0.91393119
0.40000	0.85205555	0.85214379
0.50000	0.77868986	0.77880078
0.60000	0.69759991	0.69767633
0.70000	0.61252789	0.61262639
0.80000	0.52723237	0.52729242
0.90000	0.44477638	0.44485807
1.00000	0.36783988	0.36787944



Вывод

В ходе работы я изучила методы численного решения обыкновенных дифференциальных уравнений. Программно реализовала методы Эйлера, усовершенствованного Эйлера и Милна. Исследование показало, что точность решения зависит от выбранного метода, интервала дифференцирования и шага.